Tutorial for using `Git` and `GitHub`

Huy Le Lecturer in Multiscale Materials khiem.nguyen@glasgow.ac.uk

Table of contents

Foreword 1

Installation of `Git` 1

Connect your local machine to a GitHub's repository 1

Introduction to `terminal` . . . 1

Create a GitHub's repository . . . 2

Connect to a GitHub's repository . . . 3

Get started 6

Foreword

Git and GitHub has a long history and it has been developed with in-depth and sophisticated technical features under the hood. This tutorial aims to provide readers with a direct guidance for using GitHub. The material gives step-by-step instructions by which one can apply it for their work and can have an adequate starting point for self-learning of GitHub.

Like many other software-based procedures, there are multiple ways to perform a task using GitHub. In this tutorial, we steer our focus on methods which are considered as "good practice" in terms of sustainable programming development which were suggested and recommended by the Git's user community.

Installation of `Git`

Git goes with WSL by default, so if you are using WSL or Linux operating system, you do not have to do the installation. If you are working with Windows or Mac, you need to install Git. The instruction for installation from Git's official website is straightforward, so for this point readers are directed to the website, here, to install Git.

Connect your local machine to a GitHub's repository

Introduction to `terminal`

Using Git, you will mostly do the operations via a `terminal`. So, it is worth to give a brief description of the term `terminal`. For ones who are already familiar with `terminal` can skip this part.

To have a sense of what `terminal` is, it could be described in a rudimentary way as that "*A terminal is a program that allows you to interact with your computer by typing text commands instead of clicking buttons.*"

In everyday computer use, people usually interact with a graphical interface (GUI):

- clicking folders
- dragging files
- pressing buttons

A terminal provides a text-based interface to the operating system. Instead of clicking, you type commands that tell the computer what to do. For example, to list files and folders currently in the directory, in a `terminal` (assume the `bash terminal` is used), we type:

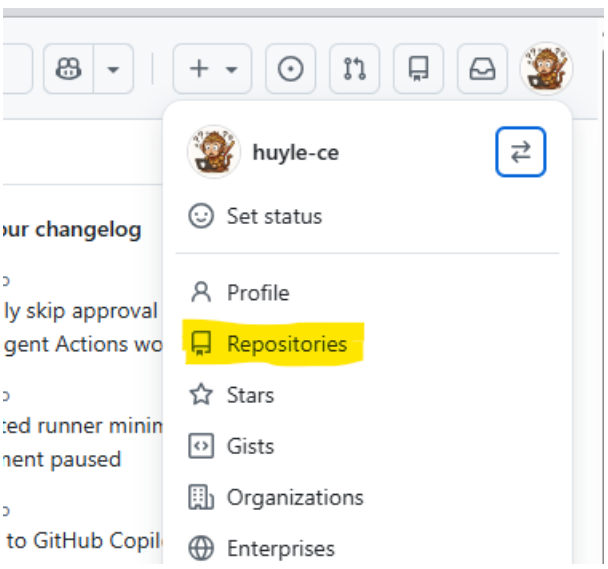
```
ls
```

and then we will see somethings like:

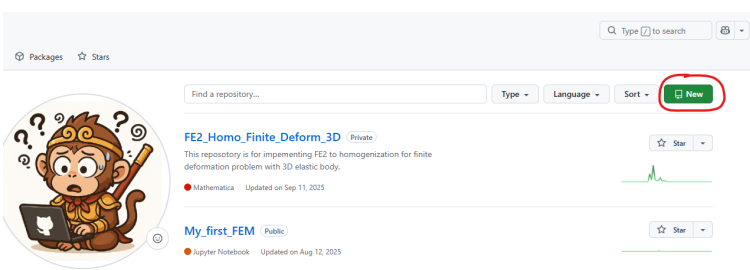
```
project
python_file.py
cpp_file.cpp
```

Create a GitHub's repository

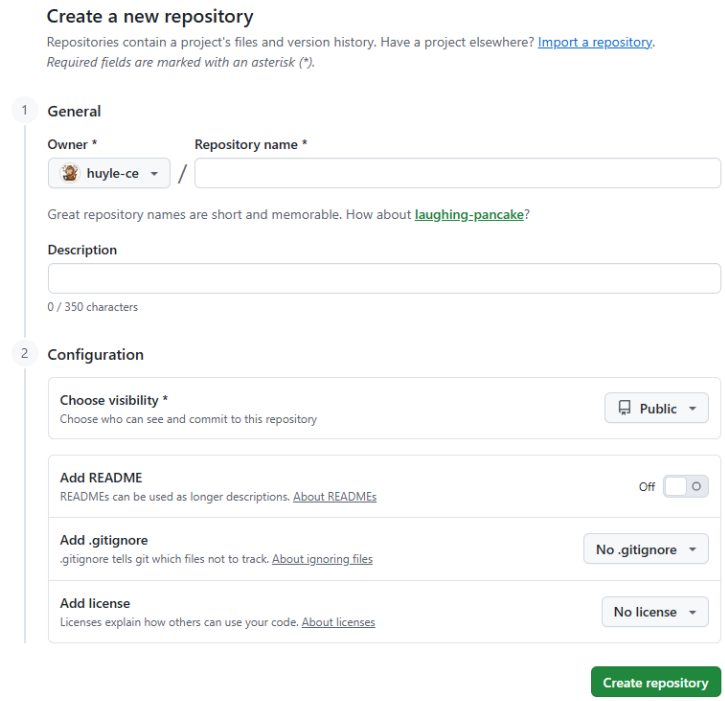
The very first thing to do is creating a repository ('repo' for short) where your project is managed. To do this, open your user navigation menu (the round icon on the top-right corner), then click `Repositories`, see the picture below:



And then, click the green `New` button:



Finally, configure your repo by filling the boxes, with the important thing being "Choose visibility" of "Public/Private". After that, click "Create repository" to create the repo

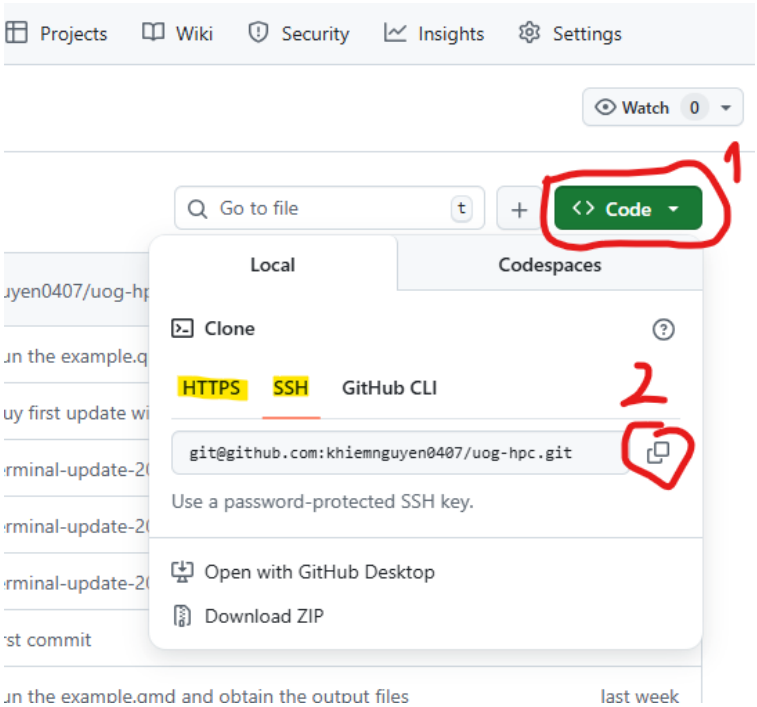


Connect to a GitHub's repository

Basically, there are two cases to get connected to a GitHub's repo, including connecting to your own GitHub repo and connect to a repo that you are invited by your collaborator. Both cases are done in a very similar way, through either `HTTPS` or `SSH` protocol.

`HTTPS` and `SSH` are protocols used by Git to communicate with remote repositories (e.g., GitHub), with the difference mainly lying in authentication, security mechanism, and workflow convenience.

Either you are connecting to your own repo or to the repo you were invited, the first thing to do is go to that repo to get the directory URL. Depending on which method you prefer, you will select `HTTPS` or `SSH` to get the link. At this point, we work through `HTTPS` and `SSH`, while leave `GitHub CLI` in later topic.



HTTPS - HyperText Transfer Protocol Secure (good for quick or temporary access)

By doing this way, when GitHub needs authentication, you will have to use "**Personal Access Token (PAT)**" (*this setup is left for you to find how to do yourself*).

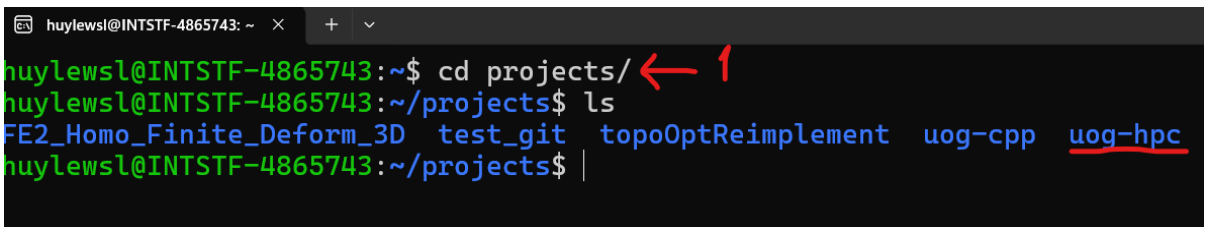
Using the copied `HTTPS` URL, you can now `clone` the repo to your local directory where your want to put your project in your local machine. In you terminal, type

```
git clone <your copied HTTPS URL>
```

For example,

```
git clone https://github.com/khiemnguyen0407/uog-hpc.git
```

To test the job, in your terminal, at the folder you keep the cloned repository, type `ls` to see whether the repo was sucessfully cloned to you local repo. See the following example of my case, first I go to my `projects` folder where I put the cloned repo of `uog-hpc`, and then `ls`.



SSH - Secure Shell (good practice and recommended)

We will go in detail of how to get connected through `SSH` protocol. It is considered as a good practice and is recommended for a long-term use of GitHub. Some of the advantages are:

- No repeated login
- Very convenient for frequent Git use
- More natural workflow for developers
- Preferred for automated systems (HPC, CI, etc.)

SSH authenticates by using the so-called "**public–private key cryptography**", which generates a "`private key`", only kept on your machine, and a "`public key`", kept on both your machine and on you GitHub account. Followings are steps to setup the SSH authentication.

This tutorial will adhere to SSH connection.

Step-by-step setup for SSH authentication:

Step 1: Generate the key: in your terminal type, and fill it with your email which you used to register your GitHub account

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

For example, my case is

```
ssh-keygen -t ed25519 -C "huy.lequang.ce@gmail.com"
```

Then, press Enter to choose the default location for storing the key (you can choose your own custom location, but we should do so unless you do have a purpose on this)

Next, enter your `passphrase` (it is like a password) to protect your `private key`. It is optional, but it provides one more layer to protect your `private key` from being exposed to someone else. (Imagine some one use your machine and open the file to have your `private key`)

Step 2: Start the SSH agent: in your terminal type

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Step 3: Copy the `public key` in order to add to your GitHub account. In your terminal, type

```
cat ~/.ssh/id_ed25519.pub
```

, and then copy the whole line.

Step 4: Add the key to your GitHub account: go to your GitHub settings (click on your account icon) –> go to "SSH and GPG keys" –> click "New SSH key" to add the key by pasting the copied content at Step 3. Note that, GitHub allows **multiple SSH keys per account**, so if you have other authentications (other machine), you can leave them untouched, and add the new SSH key.

Step 5: Test the authentication: in your terminal, type

```
ssh -T git@github.com
```

, then you should see something like:

```
Hi huyle-ce! You've successfully authenticated, but GitHub does not provide shell access.
```

Step 6: now, you can clone a repository that you have permission to clone, which can be either an invited or your own one. First, go to GitHub and copy the SSH address from the repository you want to clone, then clone it by command line:

```
git clone git@github.com:path/repository.git
```

Now, you can work with your local and GitHub repositories by pulling, pushing, and so on.

Understanding about public–private key cryptography:

After going through the procedure for setting up the SSH authentication above, it is worth to have a sense on how it works.

When doing `git clone git@github.com:user/repo.git`, the SSH protocol runs to establish a SSH channel to the GitHub's directory. Next, the SSH sends the `public key` to GitHub, and then GitHub compares this particular key with the `stored public keys`.

So, GitHub knows that this connection (or loosely speaking, this machine) is trying to authenticates as *account X* which holds the matched `public key`. Through the SSH channel, the `public key` of the account X is checked with the `private key` of the machine. If they match, the account is authenticated to be authorized by the machine, and so get connected.

When you try to clone a repository via SSH protocol, GitHub will check whether your account has permission for cloning this repository.

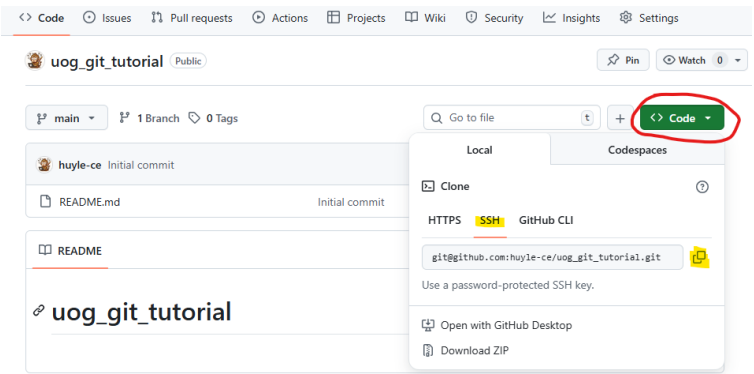
There are actually many complicated technical features behind the scene, but for a benign understanding, it can be thought of `private key` as a "key" and the `public key` as a "lock". Your machine knows both the key and the lock, while your GitHub's account holds the lock. By connecting to your GitHub account through SSH, you are asking GitHub to find the account associated with this "lock" and then use the "key" to open that "lock". If someone else can have your `public key` in their hand, and try to ask GitHub to connect themselves to your account using that `public key`, then they cannot as their machine does have the "key" - `private key` - to open the "lock".

Get started

This section aims at getting you acquainted with Git through a simple workflow where you will work with your own GitHub's repo. Now, let's put into practise what we have gone throught above.

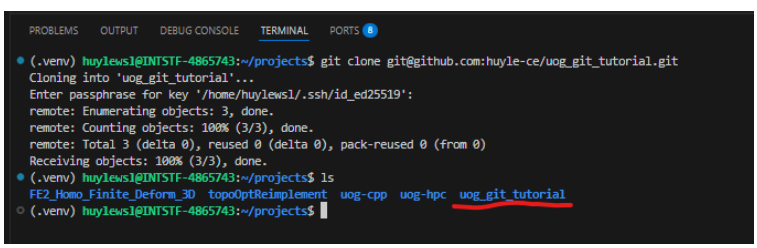
Start from `cloning` a remote repository

First, we need to clone our repo which we already created from GitHub (provide your "passphrase" if it asks you to do so):

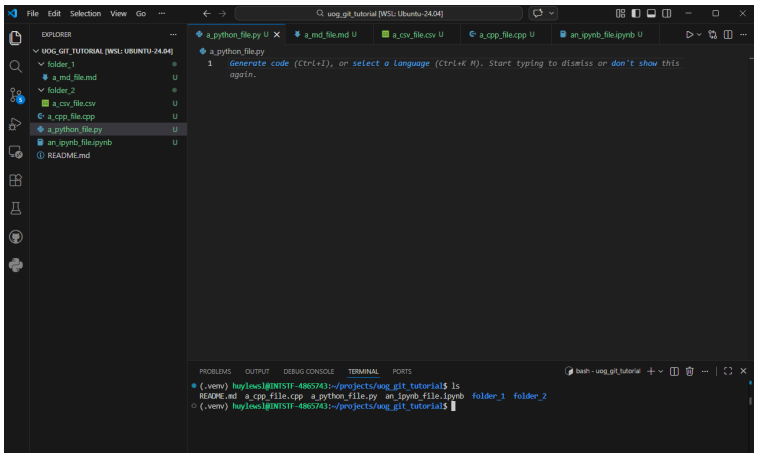



```
git clone git@github.com:huyle-ce/uog_git_tutorial.git
```

Go to your terminal , type `ls` to test whether it has done fine!



Now, we can `cd` to the repo and then type `code .` to open VS Code in this repo. It will lead you to a fresh and blank VS Code window. Hereon, you can perform tasks in exactly the same as you normally do with VS Code. Let's create some folders and files so that we can have things to do examples.



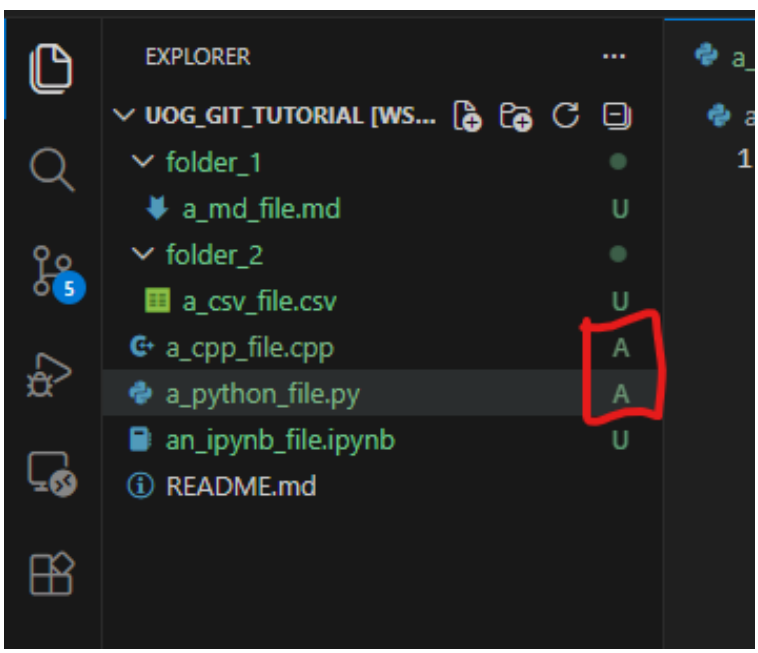
You can see the newly created folders and files appear to be green and with letter "U" on the right. This means those files are on `Working Directory` and are not tracked by `Git`, so every change and modification are basically saved on local system. Now, let us do the basic workflow step-by-step.

1. Add the files to `Staging Area`. You can add specific files or all the files.

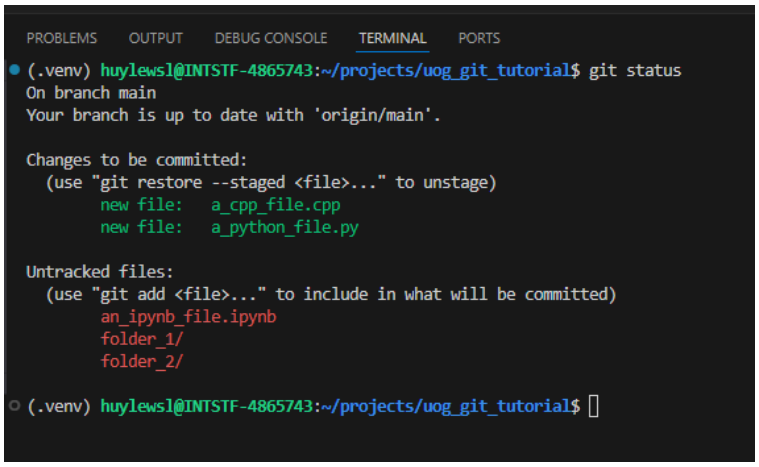
```
# Add specific files
git add a_cpp_file.cpp a_python_file.py

# Or, add all the files
git add .
```

The statuses of the added files are now changed into "A"



A handy command of a git user is `git status` which is to show the status of the current file. The following picture show the status of the files in which `a_cpp_file.cpp a_python_file.py` were added while the others were not.

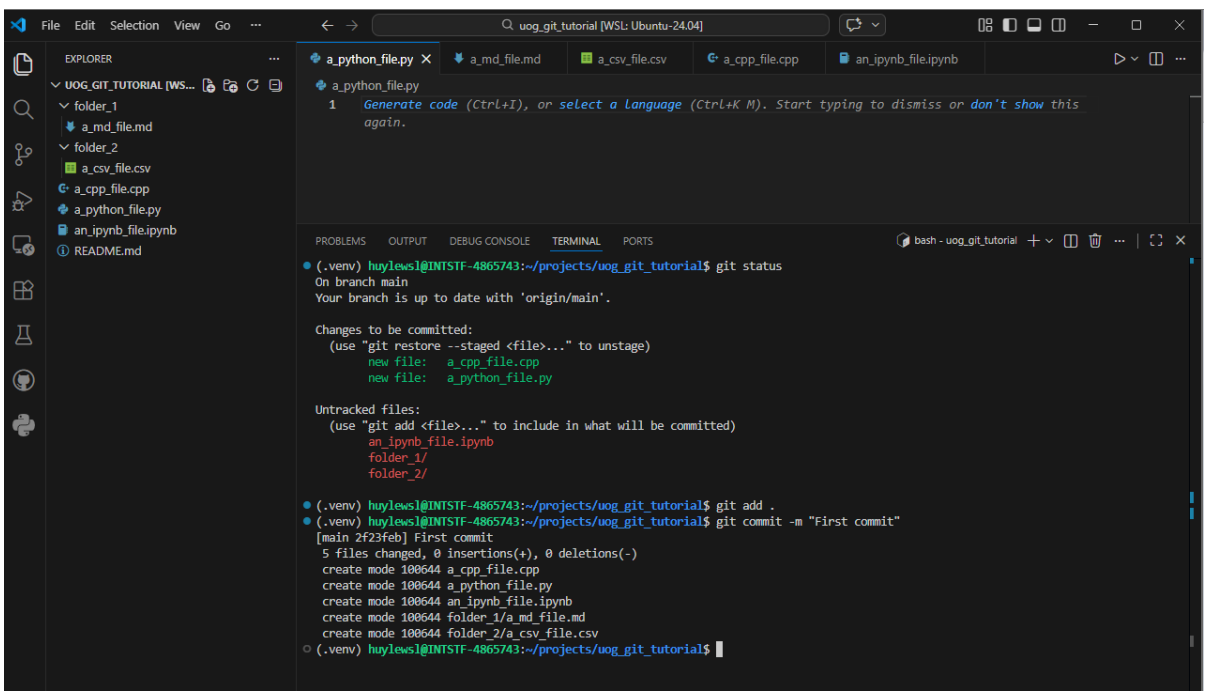


Now, we want to do it for all the files.

2. Next, we need to `commit` the added files to the so-called `Local Repository`. By doing this, Git takes everything currently in the staging area and permanently records it as a new version in the history. You can think of the `commit` action as the `save` action that you normally do when using your computer.

```
git commit -m "First commit"
```

Then you can see all the files appearing in your explorer become white and some information is provided in your terminal.



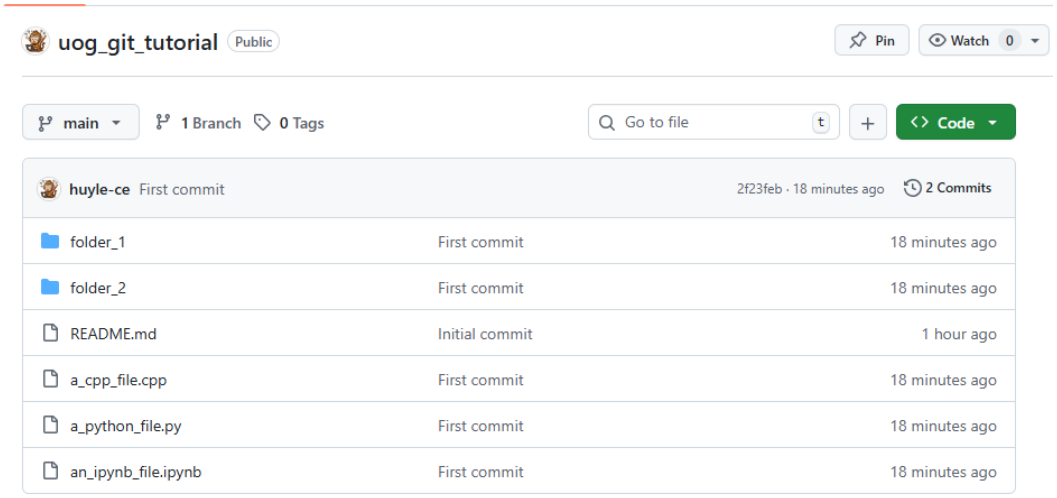
3. Up to this point, your changes are on your `main branch` and are just recorded by Git on your `Local Repository`, nothing has touched your GitHub repo. What we need to to next is `push` it to your remote repo which is essentially your GitHub's repo.

```
git push
```

What Git actually does is that it push the `main` branch to the `origin` of your remote repo. Recall that the first step is cloning the remote repo to your local machine. Through this process, a "channel" between the `main` and `origin` is established. We will go through branches in Git in detail later.

```
(.venv) huylewsl@INTSTF-4865743:~/projects/uog_git_tutorial$ git branch
* main
(.venv) huylewsl@INTSTF-4865743:~/projects/uog_git_tutorial$ git push
Enter passphrase for key '/home/huylewsl/.ssh/id_ed25519':
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 22 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (5/5), 474 bytes | 474.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:huyle-ce/uog_git_tutorial.git
   685e995..2f23feb  main -> main
(.venv) huylewsl@INTSTF-4865743:~/projects/uog_git_tutorial$
```

Now, you can revisit your GitHub repo on your web browser, refesh the page and you will see all the files were streamed to there.



Start from local repository